

SignBank Enterprise

Gesture Implementation Summary

Generated: May 20, 2026 | Project: Sign-Bank-Enterprise-mphasis

Table of Contents

- 1. Project Overview
- 2. Gesture Recognition Architecture
- 3. Two-Hand Gesture Support
- 4. Three-Finger Pinch Auto-Scroll Feature
- 5. Slider Mode (Set Limit)
- 6. Back Dynamic Gesture
- 7. Anti-Peeking Security
- 8. Gesture-to-Command Mapping System

1. Project Overview

SignBank Enterprise is a gesture-driven smart banking interaction platform. Users control all banking operations — account balance checks, card management, transaction limit settings, navigation, and logout — entirely through hand gestures captured via a standard webcam. No mouse or keyboard input is required for the primary interaction model.

Tech Stack

- **Frontend:** React 19 + TypeScript + Vite
- **Backend:** Spring Boot 4.0 + Java 21
- **Database:** PostgreSQL 16
- **Gesture Detection:** MediaPipe Hands (browser-side WASM)
- **Finger Tracking:** OpenCV via openpnp Java bindings (server-side)
- **Anti-Peeking:** MediaPipe Face Detection

Key Files

- **Frontend:** `frontend/src/hooks/useGestureControl.ts` (659 lines) — Core gesture engine
- **Frontend:** `frontend/src/components/GestureCamera/GestureCamera.tsx` — Camera UI wrapper
- **Frontend:** `frontend/src/hooks/useGlobalGestureNav.ts` — Maps gestures to navigation
- **Backend:** `OpenCvFingerTrackingService.java` — OpenCV slider tracking
- **Backend:** `GestureServiceImpl.java` — Gesture event handling

2. Gesture Recognition Architecture

All gesture recognition runs entirely in the browser using MediaPipe Hands, loaded via CDN or local WASM/TFLite models. The backend receives classified gesture events and looks up the corresponding command mappings.

Pipeline Overview

The main pipeline lives in `useGestureControl.ts` and follows this flow:

- **1. Initialization:** Opens webcam via `getUserMedia`, creates MediaPipe Hands instance with `maxNumHands: 2`, and FaceDetection instance for anti-peeking.
- **2. Main Loop:** Each animation frame calls `handsInst.send()` and `faceDetectionInst.send()` in parallel via `Promise.all`.
- **3. onResults Callback:** Extracts `multiHandLandmarks` and `multiHandedness`, draws landmarks/connections on a canvas overlay, classifies each hand.
- **4. Voting System:** Each gesture enters a rolling buffer (size=20). A gesture is confirmed when 16 out of 20 frames agree (80% threshold).
- **5. Cooldown:** Each fired gesture has a 2.2-second cooldown to prevent repeated triggering.

Gesture Types Recognized

- DIGIT (1-10) — finger count via `countFingers()`
- THUMB_UP / THUMB_DOWN — thumb position relative to base
- FIST — 0 fingers up
- OPEN_PALM — 5 fingers up
- ROCK — index + pinky up, middle + ring down
- OK — thumb tip close to index tip
- BACK_DYNAMIC — Open Palm followed by Fist held for 350ms
- SLIDER_ACTIVE / SLIDER_COMMIT — Rock gesture held for 700ms
- Three-Finger Pinch — auto-scroll activation

Finger Counting Logic

The `countFingers()` function evaluates 5 landmarks per hand:

```
function countFingers(lm: any[], label: string): number {
  let f = 0;
  // Thumb: compare x-coordinate of tip (4) vs base (3); handedness-aware
  if (label === 'Right') {
    if (lm[4].x < lm[3].x) f++;
  } else {
    if (lm[4].x > lm[3].x) f++;
  }
  // Other 4 fingers: tip y < pip y means extended
  if (lm[8].y < lm[6].y) f++; // Index
  if (lm[12].y < lm[10].y) f++; // Middle
  if (lm[16].y < lm[14].y) f++; // Ring
  if (lm[20].y < lm[18].y) f++; // Pinky
  return f;
}
```

3. Two-Hand Gesture Support

Supporting two hands enables the user to express numbers 6 through 10 (beyond what a single hand can show) and allows simultaneous independent gestures from each hand.

3.1 MediaPipe Configuration

The Hands instance is configured to track up to 2 hands simultaneously:

```
handsInst.setOptions({
  maxNumHands: 2,
  modelComplexity: 1,
  minDetectionConfidence: 0.75,
  minTrackingConfidence: 0.7,
});
```

3.2 Two-Hand Digit Summation

When 2 hands are detected, the system sums fingers from both hands to produce a combined count (1–10). This is the key innovation for two-hand support:

```
if (hands.length > 1) {
  const f0 = countFingers(hands[0], handedness[0]?.label ?? 'Right');
  const f1 = countFingers(hands[1], handedness[1]?.label ?? 'Left');
  const sum = f0 + f1;
  if (sum >= 1 && sum <= 10) {
    isTwoHandDigit = true;
    raw0 = String(sum);      // Combined count becomes the gesture
    raw1 = 'Unknown';
  }
}
```

This means: if the left hand shows 3 fingers and the right hand shows 4, the system recognizes gesture '7'. Two hands effectively extend the digit range from 1–5 to 1–10.

3.3 Two-Hand Gesture ID Mapping

Finger counts 1–5 map to G001–G005. Counts 6–10 reuse the same IDs cyclically:

```
const FINGER_TO_GESTURE_ID: Record<string, string> = {
  '1': 'G001', '2': 'G002', '3': 'G003', '4': 'G004', '5': 'G005',
  '6': 'G001', '7': 'G002', '8': 'G003', '9': 'G004', '10': 'G005',
};
```

3.4 Independent Gesture Buffers

Each hand maintains its own gesture buffer and classification pipeline. When two-hand digit mode is inactive, each hand is classified independently:

```
const handsToProcess = [
  { raw: raw0, buf: gestureBuffer0, idx: 0 },
  { raw: raw1, buf: gestureBuffer1, idx: 1 },
];
// Each hand has 3 independent state: buffer, confidence overlay,
// emoji display, and fireGesture cooldown
```

3.5 Visual Differentiation

Each hand is rendered with a distinct color and corner position:

- Hand 0 (Right): Green color #00e676, overlay at top-left

- Hand 1 (Left): Blue color #40c4ff, overlay at bottom-left
- Emoji display: H1 emoji at top-left, H2 emoji at bottom-right

The GestureCamera UI displays each hand's recognized gesture with a confidence bar:

- Single hand: ■ H1: 🖐️ ■ 95%
- Two hands: ■ H1: ■■■■ 95% and ■ H2: . . .

4. Three-Finger Pinch Auto-Scroll Feature

The auto-scroll feature allows users to scroll through pages (transaction logs, analytics, dashboards) by pinching thumb + index + middle fingertips together and moving their hand vertically. A physics engine provides smooth, natural scrolling.

4.1 Detection Logic

The system computes 3D Euclidean distances between all three finger tip landmarks (indices 4, 8, and 12 in the MediaPipe hand landmark array). When all three distances drop below 0.052 (normalized coordinate space), the pinch is activated:

```
const thumb = hands[0][4]; // Thumb tip
const index = hands[0][8]; // Index tip
const middle = hands[0][12]; // Middle tip

// 3D distances between all three finger tips
const d_thumb_index = Math.sqrt(
  Math.pow(thumb.x - index.x, 2) +
  Math.pow(thumb.y - index.y, 2) +
  Math.pow((thumb.z ?? 0) - (index.z ?? 0), 2)
);
const d_thumb_middle = Math.sqrt(...);
const d_index_middle = Math.sqrt(...);

isThreeFingerPinchActive =
  d_thumb_index < 0.052 &&
  d_thumb_middle < 0.052 &&
  d_index_middle < 0.052;
```

4.2 Physics Engine

Scroll behaves like a rate-controlled glide with heavy dampening. A separate requestAnimationFrame loop runs independently from the gesture detection loop:

```
const smoothScrollLoop = () => {
  if (isPinchLocked && cachedScrollContainer) {
    // Heavy friction dampening for jitter-free gliding
    currentVelocity = currentVelocity * 0.90 + targetVelocity * 0.10;

    // Deadband: ignore sub-0.1 velocity noise
    if (Math.abs(currentVelocity) < 0.1) currentVelocity = 0;

    currentScrollY = Math.max(0, Math.min(
      cachedScrollContainer.scrollHeight - window.innerHeight,
      currentScrollY + currentVelocity
    ));
    cachedScrollContainer.scrollTop = currentScrollY;
  }
  scrollAnimId = requestAnimationFrame(smoothScrollLoop);
};
scrollAnimId = requestAnimationFrame(smoothScrollLoop); // Starts immediately
```

4.3 Velocity Control

The pinch center's vertical offset from the screen centerline ($H/2$) determines scroll direction and speed. There is a 30px neutral deadzone at center:

```
const offset = pinchCenterY - (H / 2);
const DEADZONE = 30;

if (Math.abs(offset) < DEADZONE) {
  targetVelocity = 0; // Neutral zone – no scrolling
}
```

```

} else {
  const sign = offset > 0 ? 1 : -1; // Down = positive
  const absOffset = Math.abs(offset) - DEADZONE;
  // Curved: slow near deadzone, fast at edges
  targetVelocity = sign * Math.min(22, Math.pow(absOffset * 0.12, 1.5));
}

```

The velocity curve follows a power function (1.5 exponent), giving fine-grained control near center and faster scrolling toward screen edges.

4.4 Visual HUD

When scrolling is active, a heads-up display is drawn on the canvas:

- Horizontal dotted dividing line at screen center
- 60px translucent neutral deadzone band
- Glowing particle circle at pinch center (blue #38bdf8 for up, red #ef4444 for down)
- Rotating outer radar ring around the pinch point
- Direction indicator: ▲ GLIDING UP (75%) OR ▼ GLIDING DOWN (75%)
- Speed percentage (0–100%) shown alongside direction

4.5 Container Detection & Gesture Suppression

When pinch is active, all standard gestures are suppressed (raw0 = 'OK') to prevent accidental clicks. The scroll container is auto-detected:

```

cachedScrollContainer =
  document.getElementById('portal-content') ||
  document.querySelector('.portal-main') as HTMLElement ||
  document.documentElement ||
  document.body;

```

5. Slider Mode (Set Limit)

The slider mode allows operators to set transaction limits by holding a Rock gesture (■) for ~700ms, then sliding their hand horizontally. This is used on the Set Limit page.

5.1 Activation

```
if (raw0 === 'ROCK') {
  if (rockHoldStart === null) rockHoldStart = now;
  const held = now - rockHoldStart;
  const pct = Math.min(1, held / ROCK_HOLD_MS);
  if (held >= ROCK_HOLD_MS) {
    // Activate slider mode
    inSlider = true;
    prevSmoothedX = hands[0][0].x;
    velocity = 0;
  } else {
    drawHoldRing(ctx, cx, cy, pct); // Show progress ring
  }
}
```

5.2 Smooth Tracking

Exponential smoothing with velocity decay provides jitter-free tracking:

```
const rawX = hands[0][0].x;
const smoothX = prevSmoothedX * (1 - ALPHA) + rawX * ALPHA;
velocity = velocity * VEL_DECAY + (smoothX - prevSmoothedX) * VEL_WEIGHT;
const finalX = Math.max(0, Math.min(1, smoothX + velocity));
```

5.3 Backend Integration

The frontend sends normX (0–1) to the backend via gestureApi. The backend OpenCV finger tracking service analyzes index finger x-coordinate movement direction (LEFT/RIGHT) and speed to determine limit changes (■500–■5000 per frame).

6. Back Dynamic Gesture

A two-step gesture sequence: Open Palm (■■■) followed by Fist (■) held for 350ms triggers a navigation-back action.

```
if (raw0 === 'OPEN_PALM') {
  openPalmSeen = true;
  backStartTime = null;
} else if (openPalmSeen && raw0 === 'FIST') {
  if (!backStartTime) backStartTime = now;
  if (now - backStartTime >= BACK_HOLD_MS) {
    openPalmSeen = false;
    onGestureRef.current({ type: 'BACK_DYNAMIC', confidence: 100 });
  }
}
```

7. Anti-Peeking Security

MediaPipe FaceDetection runs alongside hand tracking. If more than 1 face is detected, a full-screen security overlay with pulsing red border locks the screen until the extra person leaves. Overlay features:

- Fixed position, 100vw x 100vh, z-index 999999

- Blurred dark backdrop with red pulsing glow animation
- Warning text: 'SCREEN PEEKING DETECTED!'
- Auto-dismisses when face count returns to 1

8. Gesture-to-Command Mapping System

An admin panel at `/admin/mappings` maps gesture IDs (G001–G009) to commands per role. The backend stores these in a `CommandMapping` JPA entity. Seed data includes:

Gesture ID	Finger Count	Command	Role
G001	1 Finger / 6 Fingers	Check Balance	operator
G002	2 Fingers / 7 Fingers	Logout	operator
G003	3 Fingers / 8 Fingers	Back	operator
G004	4 Fingers / 9 Fingers	View Logs	viewer
G005	5 Fingers / 10 Fingers	View Analytics	viewer

This document summarizes the gesture implementation in SignBank Enterprise. The core gesture engine lives in `useGestureControl.ts` (659 lines) with support from `GestureCamera.tsx`, `useGlobalGestureNav.ts`, and the backend `GestureService`. All gesture recognition runs entirely client-side via MediaPipe Hands WASM.